

2장 Variables and Data Structure

변수

type(var) 변수의 타입 확인
var1=var2=var3=var4=100 가능

데이터 표현

bit : 최소단위
8bit == 1byte

hex bin dec oct

dec -> bin 2로 계속 나누고 역순으로 나열하기!

```
bin(11)      #'0b1011'  
oct(11)      #'0o13'  
hex(11)      #'0xb'  
bin(0o11)    #'0b1001'
```

int(<문자열 형식의 변수>, <base>) #base진수 문자열을 dec로 변환해라
int('1010', 2) # 2진수 '1010' → 10진수로 10
int('17', 8) # 8진수 '17' → 10진수로 15
int('1A', 16) # 16진수 '1A' → 10진수로 26

```
sel = int(input("Select base (16/10/8/2) :"))  
num = input("Enter the value : ")  
num = int(num, sel)  
print("Hexadecimal : ", hex(num))  
print("Decimal : ", int(num))  
print("Octal : ", oct(num))  
print("Binary : ", bin(num))
```

_____결과_____

```
Select base (16/10/8/2) :8  
Enter the value : 22  
Hexadecimal : 0x12  
Decimal : 18  
Octal : 0o22  
Binary : 0b10010
```

자료형

숫자형 : 자연수
int형 : 크기 무제한! base(진수)변환 지원
실수형 : 소수점 잇는 모든 수 지원 실수는 너무 크면 에러!

산술연산

+ - * / **제곱, %나머지, //몫

bool 타입

```
a = (100 == 100)    # True
True or False
```

String

따옴표를 출력하고 싶으면 다른 따옴표 쓰기!

또는 \' \' 쓰기

```
a = 'I\'d like to go'
```

이스케이프 문자

\n 줄바꿈

따옴표 뒤에 % 하고 printf처럼 출력

%f 실수형

%d 정수형

```
x = 100.4
```

```
print("The value of x is %f" % x)
```

```
print("%d / %d = %d" % (100, 200, 0.5))
```

```
print("%05d" % 123) # 00123 5자리 만들어놓고 앞에는 0으로 채우기
```

```
print("%5d" % 123) # 123 5자리 만들어놓고 출력
```

```
print("%d" % 1234) # 1234 그냥 출력
```

```
print("%7.1f" % 123.45) # 123.5 7자리 만들어놓고 소수점 1번째자리까지 출력(반올림)
```

```
a=1.5
```

```
b=2.5
```

```
c=3.5
```

```
d=4.5
```

```
print("%5.0f %5.0f %5.0f %5.0f" % (a,b,c,d))
```

3장 Operators

이런 식으로 초기화도 가능! a, b, c = 2, 3, 4

	//	%	확인
5//2	2	1	5 = 2 * 2 + 1
-5//2	-3	1	-5 = -3 * 2 + 1
5//-2	-3	-1	5 = -2 * -2 -1
-5//-2	2	-1	-5 = 2 * -2 + -1

비트연산

```
10 & 7 # 2
```

```
10 | 7 # 15
```

```
10 ^ 7 # 13 XOR
```

```
~367 +1 # -367 내부적으로 ~x 는 -x-1이다.
```

4장 If structure

if elif else문

ternary operator 삼항연산자

```
score = 70
res = 'pass' if score >= 60 else 'fail'
print(res)    #pass
```

eval() 연산식 계산함수

```
eval('12/4')          #3.0    계산해서 결과가 나옴!
eval(equation)         #239.28571428571428
```

리스트

```
fruit = ['apple', 'pear', 'strawberry', 'grape']
print(fruit)          #['apple', 'pear', 'strawberry', 'grape']
fruit.append('melon')
print(fruit)          #['apple', 'pear', 'strawberry', 'grape', 'melon']

'strawberry' in fruit    #'strawberry'가 fruit 리스트 안에 있는가?
```

```
if 'strawberry' in fruit:
    print("Here's strawberry!")
```

```
import random
numbers = []
for num in range(0, 10):
    numbers.append(random.randrange(0, 10))
print("Created list", numbers)
for num in range(0, 10):
    if num not in numbers:
        print("Number %d is not in the list." % num)
```

5장 For structure

```
for i in range(2, -1, -1):
    print("%d " % i)          # 2, 1, 0

for i in range(1, 6, 1):
    print("%d " % i, end=" ")    #1, 2, 3, 4, 5

for i in range(0, 3, 1):
    for k in range(0, 2, 1):
        print("i : {}, k : {}".format(i, k))
```

```
# i : 0, k : 0
# i : 0, k : 1
# i : 1, k : 0
# i : 1, k : 1
# i : 2, k : 0
# i : 2, k : 1
```

```
i = 1
result = 0
while i<11:
    result += i
    i += 1
print(result)
while True
```

6장 List

```
aa = [10, 20, 30, 40]
aa[-2]      # 30
```

```
aa = [[10, 20], 30]
aa[0]      # [10,20]           리스트 안 리스트도 가능!
```

```
aa = [10, 20, 30, 40]
aa[3]      # IndexError       인덱스 오류
```

```
myList=[30, 10, 20]
myList.append(40)    #맨 뒤에 추가
myList.pop()         #맨 뒤엿것 pop(빼고 return)
```

```
myList.sort()        #[10, 20, 30]   정렬하기
newList=sorted(myList)    #myList자체가 바뀌지는 않고 리턴해줌!
myList.reverse()     #[30, 20, 10]   역순으로 정렬하기
```

```
myList.index(20)     #1           '20'이라는 데이터가 있는 원소의 index를 리턴
```

```
myList.insert(2, 222)    #2번째 idx에 222라는 데이터를 추가하겠다
myList                  #[30, 20, 222, 10]
```

```
myList.remove(222)     #222라는 데이터를 찾아 삭제하겠다.
```

```
myList.extend([77, 88, 77])    #맨 뒤에 리스트를 이어붙인다.
myList                        #[30, 20, 10, 77, 88, 77]
```

```
myList.count(77)       #2           77이라는 데이터가 몇 번 들어있는지 알려줌
```

인덱싱, 슬라이싱

```
aa = [10, 20, 30, 40]
aa[-1]      #40
aa[0:3]     #[10, 20, 30]
```

```

aa[2:4]      #[30,40]
aa[2:]       #[30,40]
aa[:2]       #[10,20]
aa[::2]      #[10,30]      점프를 2로!
aa[::-2]     #[40,20]      맨 뒤엣것부터 기준으로 점프를 -2!
aa.reverse() #[40,30,20,10] 역순정렬

```

```

aa = [10, 20, 30]
bb = [40, 50, 60]
aa+bb        #[10,20,30,40,50,60]    ★원소를 더하는 것 아님주의!!! 단순히 리스트 이어붙이기 ★

```

```

len(aa)      #3      리스트의 길이를 출력

```

```

list1 = []
list2 = []
value = 1
for i in range(0, 3):
    for j in range(0, 4):
        list1.append(value)
        value += 1
    print(list1)
    list2.append(list1)
    list1 = []

```

_____결과_____

```

[1, 2, 3, 4]
[5, 6, 7, 8]
[9, 10, 11, 12]

```

```

list1 = []
list2 = []
value = 0
for i in range(0, 4):
    for j in range(0, 5):
        list1.append(value)
        value += 3
    list2.append(list1)
    list1 = []
print(list2)
[[0, 3, 6, 9, 12],
 [15, 18, 21, 24, 27],
 [30, 33, 36, 39, 42],
 [45, 48, 51, 54, 57]]

```

튜플

```

tt1 = (10, 20, 30)
tt1.append(40)      #AttributeError 튜플은 추가할 수 없다!
del(tt1[0])         #삭제도 안 된다!
del(tt1)            #튜플 객체 자체를 없애버림!! 이 다음줄에서 tt1을 찾으려고 하면 NotDefined 에러가 뜸

```

```

tt1 = (10, 20, 30, 40, 50)
tt2 = ('A', 'B')
tt1+tt2      #(10, 20, 30, 40, 50, 'A', 'B') 연산 자체는 List와 동일!

```

```

myTuple = (10, 20, 30)
myList = list(myTuple)
myList.append(40)
myTuple = tuple(myList)
print(myTuple)          #리스트를 튜플로 형변환 가능!

```

딕셔너리 Dictionary

```

dic1 = {1:'a', 2:'b', 3:'c'}      #{키:값, 키:값}
print(dic1)

```

```

student1 = {'ID' : 1000, 'Name' : 'Hong Gil-dong', 'Department' : 'Computer Science'}
student1      #딕셔너리가 전체가 출력됨
student1['Contact'] = '010-1111-2222'      #Contact라는 데이터가 추가됨.

```

```

student1['Department']      #'Computer Science'      Key로 접근
student1['Department'] = 'Business'      Key로 수정
del(student1['Department'])      #Department라는 키:값 삭제

```

```

student1 = {'ID' : 1000, 'Name' : 'Hong Gil-dong', 'Department' : 'Computer Science', 'ID' : 2000}
      #중복되는 레코드가 있으면 뒤엎 것으로 적용됨!  키 중복 불허!
student1['ID']      #2000
student1.get('ID')  #2000      이것 또한 값을 불러옴.

```

```

그런데, 키가 존재하지 않을때는 뭘 방법은 error를 내지만, .get()은 아무것도 리턴하지 않음.
student1['address']      #KeyError: 'address'
student1.get('address')  #아무것도 리턴 안함

```

```

student1.keys()      #dict_keys(['ID', 'Name', 'Department'])      키를 전부 출력
>> 그런데 이걸 list로 사용하려면 형변환을 해 줘야 함.
list(student1.keys())

```

```

student1.values()      #dict_values([2000, 'Hong Gil-dong', 'Computer Science'])

```

```

student1.items()      #dict_items([('ID', 2000), ('Name', 'Hong Gil-dong'), ('Department', 'Computer Science')])
      #키:값을 튜플이 담긴 리스트로 리턴

```

```

'Name' in student1      #True      Name이라는 키가 있니? > True/False로 리턴
'Address' in student1      #False

```

```

singer={}
singer['Name'] = 'IU'
singer['Age'] = 30

```

```
singer['Song'] = 'Good Day'
singer['Debut'] = 2008
```

```
for k in singer.keys():                #for문 쓸때는 굳이 list로 형변환 안해줘도 됨! 해도됨....
    print(k, ":", singer[k])
```

```
import operator
```

```
trainDic = {}
trainList = []
trainDic = {'Thomas':'토마스', 'Edward':'에드워드', 'Henry':'헨리', 'Gothen':'고든', 'James':'제임스'}
trainList = sorted(trainDic.items(), key=operator.itemgetter(0))
    #tainDic.items() 튜플에서 0번째 — 즉, 키값을 기준으로 정렬하라는 의미.
print(trainList)                #[('Edward', '에드워드'), ('Gothen', '고든'), ('Henry', '헨리'), ('James', '제임스'), ('Thomas', '토마스')]
```

```
a = [4,1,1,3,1,2]
```

```
answer = {}
```

```
for i in range(0, len(a)):
    if a[i] in answer:
        answer[a[i]] += 1
    else:
        answer[a[i]]=1
```

```
print(answer)                    #{4: 1, 1: 3, 3: 1, 2: 1}
```

7장 String

```
ss = "Python"
ss[0]      #'P'
ss[1:3]    #'yt'
ss = 'Python' + 'Class'
ss        #'PythonClass'
```

```
len("abcd") #4
```

```
ss = 'test'
for i in range(0, len(ss)):
    print(ss[i]+"$", end='')        #t$e$s$t$
```

```
ss = 'test'
for c in ss:
    print(c+"$", end='')            #t$e$s$t$
```

```
ss = input("Enter the string: ")
for i in range(len(ss)):
    print(ss[-i-1], end="")          #역순으로 출력
```

```
ss = input("Enter the string: ")
print(ss[::-1])                     #마찬가지로 역순으로 출력
```

```
ss = "Python is Easy."
ss.upper()          #'PYTHON IS EASY.'      모두 대문자로
ss.lower()          #'python is easy.'      모두 소문자로
ss.swapcase()       #'pYTHON IS eASY.'      대문자->소문자, 소문자->대문자
ss.title()          #'Python Is Easy.'      단어의 첫번째만 대문자로
```

```
ss = "Studying python is fun. It is true."
ss.count('is')      #2                  is가 몇번 들어있나?
ss.find("is")       #16                 is가 몇번째 idx에 들어있나?
ss.rfind("is")      #27                 오른쪽부터 찾아서 is가 몇번째 idx에 들어있나?
ss.find("is", 20)   #27                 20번째 이후로 is를 찾아라
ss[27]             #'i'                27번째 idx 인덱싱
ss.find("are")      #-1                 없는거 찾으려면 -1
ss.index("are")     #ValueError         없는 index를 찾으려면 ValueError.
```

```
ss.startswith("python")    #False      python으로 시작하니?
ss.startswith("Stu")       #True       Stu로 시작하니?
ss.endswith("Stu")         #True       .로 끝나나?
ss.startswith("python",9)  #True       9번째 idx부터 python으로 시작하니?
ss[9:].startswith("python") #True      9번째 idx부터 python으로 시작하니?
```

```
ss = "      dfd      fdf      "
ss.strip()      #'dfd      fdf'      앞뒤 공백제거
ss.rstrip()     #'      dfd      fdf' 뒤(오른쪽) 공백제거
ss.lstrip()     #'dfd      fdf      ' 앞(왼쪽) 공백제거
```

```
ss = '-----Pyt---hon-----'
ss.strip('-')    #'Pyt---hon'        '-'글자를 앞뒤로 제거
ss = 'aaaaaabbbbccccaaa'
ss.strip('a')    #'bbbbcccc'
```

```
inStr = "   Python   Programming   "
outStr = ""
for i in range(len(inStr)):
    if inStr[i] != ' ':                #공백이 아니면 outStr에 추가!
        outStr += inStr[i]
print(outStr)                         #PythonProgramming
```

```
inStr = "<<<<Py<<th>>>>on>>>"
outStr = ""
for i in range(len(inStr)):
    if inStr[i]!='<' and inStr[i] != '>':    #<과 >가 아니면 outStr에 추가
```



```
outStr += inStr[i]
print(outStr)
```

```
ss = "Lakers won the NBA championship"
ss.replace("Lakers", "Lebron")      #문자열에서 앞엿걸 찾아서 뒤엿걸로 바꿔라.
```

```
ss = "A new line"
ss.split()      #['A', 'new', 'line']      스페이드 단위로 쪼개기
```

```
ss = 'One:two:three'
ss.split(':')    #['One', 'two', 'three']      : 단위로 쪼개기
```

```
ss = 'bbbaccadd'
ss.split('a')    #['bbb', 'ccc', 'ddd']      a를 기준으로 쪼개기
```

```
ss = 'One\ntwo\nthree'
ss.splitlines()  #['One', 'two', 'three']      줄을 기준으로 쪼개기
```

```
ss = '%'
ss.join('python')    #'p%y%t%h%o%n'      글자 사이사이에 집어넣기
```

```
ss = input("Input date: ")
ssList = ss.split('/')
year = int(ssList[0])
month = ssList[1]
day = ssList[2]
print("Year"+str(year+10) + "Month" + month + "Day" + day)
```

```
# Input date: 2019/12/31
# Year2029Month12Day31
```

```
before = ['2019', '12', '31']
after = list(map(int, before))      #before iterable(리스트)에 int()함수를 적용해라
after      #[2019, 12, 31]
```

```
ss = 'python'
ss.center(10)      # ' python '      10칸만큼 자리를 만들어놓고 가운데정렬
ss.center(10, '-')  # '--python--'    10칸만큼 자리를 만들어놓고 가운데정렬. 빈칸은 '-'로
ss.ljust(10)        # 'python '      왼쪽정렬
ss.rjust(10)        # ' python'      오른쪽정렬
ss.zfill(10)        # '0000python'    오른쪽정렬 후 나머지 0으로 채우기
```

```
'1234'.isdigit()    #True
'12a456'.isdigit()  #False
'abcd'.isalpha()    #True
'abc1'.isalpha()    #False
'ab c'.isalpha()    #False
'abc.'.isalpha()    #False
```

```
'abc123'.isalnum()      #True
'abc 123'.isalnum()     #False
'abcd'.islower()       #True
'abcd1'.islower()      #True
'ABCD'.isupper()       #True
'   '.isspace()        #True
```

```
inStr = input("Enter a string: ")
if inStr.isalpha():
    print("It is a letter")
elif inStr.isdigit():
    print("It is a number")
elif inStr.isalnum():
    print("It is a letter + number")
else:
    print("I don't know")
```

```
import operator
```

```
inStr = input("Enter a string: ")      #bbaaac
countdict = {}
for c in inStr:                        #STRING에 뭐가 몇번 들어있는지 count해서 dict에 저장
    if c in countdict:
        countdict[c] += 1
    else:
        countdict[c] = 1

countlist = sorted(countdict.items(), key=operator.itemgetter(1))
print(countlist)                       #abc
for i in range(len(countlist)):
    print(countlist[-i-1][0], end='')   #['c', 1), ('b', 2), ('a', 3)]
```

9장 Function and Module

```
def coffee_machine(button):
    print("#1. Prepare hot water")
    print("#2. Prepare a cup")

    if button == 1:
        print("#3. Make normal coffee")
    elif button == 2:
        print("#3. Make sugar coffee")
```

```
def plus(v1, v2):
    result = v1 + v2
    return result
res = plus(100, 200)
print(res)                        #300
```

```
def calc(v1, v2, op):
    result = 0
    if op=='+':
        result = v1 + v2
    elif op=='-':
        result = v1 - v2
    elif op=='*':
        result = v1*v2
    elif op=='/':
        result = v1/v2
    return result
```

```
calc(10, 20, '*')
```

```
calc(v1=10, op='*', v2=20)
```

#직접 지정해서 파라미터 전달

```
calc(10, 0, '/')
```

#0으로 나누면 에러남!

```
>> elif op=='/':
```

#이런식으로 수정해야 함

```
    if v2 != 0:
```

```
        result = v1/v2
```

```
    else:
```

```
        print("Cannot divide by 0")
```

```
    return
```

```
def func1():
```

```
    a = 10 #local
```

```
    print("Value of a in func1(): %d" % a)
```

```
def func2():
```

```
    print("Value of a in func2(): %d" % a)
```

```
a = 20 #global
```

```
func1()          # 10
```

local이 global보다 우선이다!

```
func2()          # 20
```

```
def func1():
```

```
    global a
```

```
    a = 10
```

```
    print("Value of a in func1(): %d" % a)
```

```
def func2():
```

```
    print("Value of a in func2(): %d" % a)
```

```
a = 20
```

```
func1()          #global a 하면 전역변수취급! 안에서 수정한게 바깥까지 바뀐다.
```

```
func2()
```

```
def func2():
```

```
    print("Run function without return value")
```

```
b = func2()
```

```
print(b)
```

#None

return이 없는 함수 자체를 출력하면 None이 나온다.

```
def multi(v1, v2):
    ret_list = []
    res1 = v1+v2
    res2 = v1-v2
    ret_list.append(res1)
    ret_list.append(res2)

    return ret_list
my_list = multi(100, 200)
my_list          #[300, -100]    리스트를 return 할 수도 있다! 리스트 형태로 리턴됨.
```

```
def para_func(v1, v2, v3=0):          #가능!
    result = v1+v2+v3
    return result

def para_func(v1, v2=0, v3):          #불가능!!
    result = v1+v2+v3                #기본값이 있는 파라미터는 기본값이 없는 파라미터 앞에 올 수 없음!
    return result
```

```
def para_func(*args):
    result = 0
    print(args)
    for num in args:
        result += num
    return result

a = para_func(1,2,3,4,5,4,6,3,7,5,3,2,1,5,)    #(1, 2, 3, 4, 5, 4, 6, 3, 7, 5, 3, 2, 1, 5)
print(a)                                         #51
```

```
def dic_func(**kwargs):                #전달되는 파라미터를 딕셔너리로 묶어줌!
    print(kwargs)
    for k in kwargs.keys():
        print("%s : Back number %d" % (k, kwargs[k]))

dic_func(Lebron=23, Kobe=24, Paul=3)          #정상!
dic_func('Lebron'=23, 'Kobe'=24, 'Paul'=3)    #에러!! 따옴표를 쓰면 안된다!!!
출력
{'Lebron': 23, 'Kobe': 24, 'Paul': 3}
Lebron : Back number 23
Kobe : Back number 24
Paul : Back number 3
```

```
_____모듈에서 함수 불러오기_____

import Module1
Module1.func1()
Module1.func2()
Module1.func3()

from Module1 import func1, func2, func3
from Module1 import *          #전부 다 불러오기
```

```
import math
import random
import os
import sys
```

```
print(sys.builtin_module_names) #함수이름 짝 뜸
dir(math)                       #애도 함수이름 짝뜸
```

```
import time
dir(time)                       #애도 마찬가지로...
```

```
print("A")
time.sleep(5)                   5초 후에 사용
print("B")
```

```
import numpy
import numpy.random
from numpy.random import random
from numpy.random import *
```

```
def outFunc(v1, v2):           #내부 함수
    def inFunc(num1, num2):
        return num1+num2
    return inFunc(v1, v2)
```

```
outFunc(10, 20) #30
inFunc(10, 20)  #바로 inFunc를 호출하는 것은 불가능.
```

```
add2 = lambda num1, num2 : num1+num2    #람다 함수
add2(10, 20)    #30
```

```
add3 = lambda num1=10, num2=20 : num1+num2
add3()    #30
```

_____람다를 안쓴 코드_____

```
my_list = [1,2,3,4,5]
def add10(num):
    return num+10
for i in range(len(my_list)):
    my_list[i] = add10(my_list[i])
print(my_list)
```

_____람다를 쓴 코드_____

```
my_list = [1,2,3,4,5]
add10 = lambda num : num+10
my_list = list(map(add10, my_list))    #my_list에 add10함수를 적용해라!
print(my_list)
```

_____람다를 쓴 또다른 코드_____

```
my_list = [1,2,3,4,5]
my_list = list(map(lambda num : num+10, my_list))    #my_list에 람다 함수를 적용해라!
print(my_list)
```

```
list1 = [1,2,3,4]
list2 = [10,20,30,40]
sum_list = list(map(lambda n1, n2 : n1+n2, list1, list2)) #list1과 list2 iterable에 n1+n2를 적
용해라!
```

```
print(sum_list)
```

_____Recursive call 재귀함수_____

```
def count(num):
```

```
    if num >= 1:
```

```
        print(num, end=' ')
```

```
        count(num-1)
```

```
    else:
```

```
        return
```

```
count(10) #10 9 8 7 6 5 4 3 2 1
```

_____factorial 재귀함수_____

```
def factorial(num):
```

```
    if num <= 1:
```

```
        return num
```

```
    else:
```

```
        return num * factorial(num-1)
```

```
factorial(4) #24
```

_____Generator와 yield_____

Return 대신 yield를 쓰면 **Generator 함수**가 된다.

함수를 호출한다고 해서 함수 안이 바로 실행되지 않음.

```
def genFunc():
```

```
    yield 1 #첫번째 값인 1 생성
```

```
    yield 2 #두번째 값인 2 생성
```

```
    yield 3 #세번째 값인 3 생성
```

```
list(genFunc())
```

```
for n in genFunc():
```

```
    print(n) #1 \n
```

```
import time
```

```
def genFunc2():
```

```
    for n in [1,2,3]:
```

```
        yield n
```

```
        time.sleep(1)
```

```
for n in genFunc2():
```

```
    print(n) #출력이 1나오고 1초있다가 2나오고 1초 있다가 3나옴!
```

```
print(list(genFunc()))
```

```
# 실행 결과: [1, 2, 3]
```

이 코드가 실행되는 과정은 다음과 같습니다.

1. `genFunc()` 가 호출되어 제너레이터 객체(실행 대기표)가 생성됩니다.
2. `list()` 함수는 이 제너레이터 객체에게 "가지고 있는 모든 값을 순서대로 다 주세요"라고 요청합니다.
3. 제너레이터는 실행을 시작하여 첫 번째 `yield 1` 을 만납니다. 값 `1` 을 `list()` 에게 넘겨주고 실행을 일시 정지합니다. (이때 함수는 종료되지 않고, 어디까지 실행했는지 기억합니다.)
4. `list()` 는 다음 값을 또 요청합니다. 제너레이터는 멈췄던 곳 다음부터 실행을 재개하여 두 번째 `yield 2` 를 만납니다. 값 `2` 를 넘겨주고 다시 일시 정지합니다.
5. `list()` 가 마지막 값을 요청합니다. 제너레이터는 다시 실행을 재개하여 세 번째 `yield 3` 을 만나고 값 `3` 을 넘겨줍니다.
6. 함수 끝까지 실행되어 더 이상 `yield` 할 값이 없으면 제너레이터는 종료됩니다.
7. `list()` 는 그동안 받은 값 `1, 2, 3` 을 모아 리스트 `[1, 2, 3]` 을 만들고, `print` 함수가 이를 화면에 출력합니다.

▪ [Exercise]

- Write code that returns a new list by multiplying the corresponding elements of two lists, `a_list` and `b_list`, which have the same length.

```
a_list = [1,2,3,4,5]
```

```
b_list = [5,6,7,8,9]
```

```
def multiply(n1, n2):
```

```
    return n1*n2
```

```
new_list = []
```

```
for i in range(len(a_list)):
```

```
    new_list.append(a_list[i] * b_list[i])
```

```
print(new_list)
```

```
_____람다 사용_____
```

```
a_list = [1,2,3,4,5]
```

```
b_list = [5,6,7,8,9]
```

```
new_list = list(map(lambda n1, n2 : n1*n2, a_list, b_list))
```

```
print(new_list)
```

10장 File Input Output

```
variable_name = open("file_name", "r")
```

```
#파일 열기. 읽기
```

```
variable_name = open("file_name", "w")
```

```
#파일 열기. 쓰기
```

```
inFp = None
```

```
#파일포인터
```

```
inStr = ""
```

```
#Step 1 열기
```

```
inFp = open("data/data22/data1.txt", "r")
```

```
# Step 2 Process하기
```

```
inStr = inFp.readline() #readline()을 할때마다 포인터가 다음으로 이동!
```

```
print(inStr, end="")
```

```
inStr = inFp.readline()
```

```
print(inStr, end="")
```

```
inStr = inFp.readline()
```

```
print(inStr, end="")
```

```
# Step 3 닫기
```

```
inFp.close()
```

```

inFp = open("C:/Users/CAU/software_programming/Class_7/data1.txt", "r")
for i in range(3):
    inStr = inFp.readline()                #이런식으로 반복문으로 읽어들이어야 함. inFp는 자동으로
    print(str(i+1) + ": " + inStr, end="")
inFp.close()

```

1: Python Class

2: Week 10

3: File I/O

```

inFp = open("data1.txt", "r")
inList = inFp.readlines()                #readlines() 여러 줄 읽어옴
print(inList)                            #['Python Class\n', 'Week 10\n', 'File I/O']
inFp.close()                             #readlines는 \n까지 같이 출력함!

```

```

inFp = open("data1.txt", "r")
inList = inFp.readlines()
for inStr in inList:
    print(inStr, end="")
inFp.close()

```

Python Class

Week 10

File I/O

```

with open("data1.txt", "r") as inFp:      #scope가 끝나면 파일을 자동으로 닫아줌
    inList = inFp.readlines()
    print(inList)                         #['Python Class\n', 'Week 10\n', 'File I/O']
#    inFp.close()                         #닫을 필요 없음!

```

```

import os
os.path.exists("C:/Windows/win.ini")      #True 이 파일이 존재하면 True 아니면 False 반환

```

```

fName = input("File name: ")
if os.path.exists(fName):                 #파일이 존재하면
    inFp = open(fName, "r")
    inList = inFp.readlines()
    for inStr in inList:
        print(inStr, end="")
    inFp.close()
else:                                     #파일이 존재하지 않으면
    print("%s does not exists." % fName)

```

```

# Step 1
outFp = open("data2.txt", "w")
# Step 2
while True:
    outStr = input("Write content: ")
    if outStr != "":
        outFp.writelines(outStr + "\n")
    else:
        break
# Step 3
outFp.close()

```

```

inFp = open("C:/Windows/win.ini", "r")
outFp = open("C:/Temp/data3.txt", "w")
inList = inFp.readlines()
for inStr in inList:
    outFp.writelines(inStr)
inFp.close()
outFp.close()
print("File copied")

```

odr('C')	#65	ord() ordinal 함수 : 글자를 숫자로
ord('D')	#68	
ord('E')	#69	
ord('안')	#50504	
ord('^')	#94	
chr(67)	#'C'	chr() character 함수 : 숫자를 글자로
chr(50504)	#'안'	
chr(ord('G')+100)	#'«'	
chr(ord('o')+100)	#'Ó'	

```

origin = "Go"
encrypted = ""
for i in range(len(origin)):
    encrypted += chr(ord(origin[i])+100)
print(encrypted)

```

```

decrypted = ""
for i in range(len(encrypted)):
    decrypted += chr(ord(encrypted[i])-100)
print(decrypted)

```

```

option = int(input("1. Encryption  2. Decryption  Select: "))
in_fname = input("Enter input file name: ")
out_fname = input("Enter output file name: ")

```

```

in_file = open(in_fname, "r", encoding="utf-8")
out_file = open(out_fname, "w", encoding="utf-8")

```

```

if option==1:
    key_val = 100
else:
    key_val = -100
while True:
    in_str = in_file.readline()
    if not in_str:
        break
    out_str = ""
    for i in range(len(in_str)):
        out_str += chr(ord(in_str[i]) + key_val)
    out_file.writelines(out_str)
out_file.close()
in_file.close()

```

_____binary 입출력_____

binary 파일이란? : 텍스트로 이루어져있지 않은 다른 모든 파일. 영상, 이미지, 음악, 엑셀, exe 등등....

```

inFp = open("C:/Windows/notepad.exe", "rb")
outFp = open("C:/Temp/notepad.exe", "wb")
while True:
    inStr = inFp.read(1)      #1바이트씩 읽기
    if not inStr:            #inStr이 거짓이면!
        break
    outFp.write(inStr)
inFp.close()
outFp.close()
print("Binary file copied")

```

```

import os
from PIL import Image

file_path = "lena.raw"
width = 256; height = 256
with open(file_path, 'rb') as f:
    raw = f.read()          #파일 끝까지 한번에 읽어버림.
flat=list(raw)             #raw를 리스트로 만들어 버림
pixels = [flat[row_start:row_start+width] for row_start in range(0, width*height, width)]
img = Image.new('L', (width, height))      #'L'은 그레이스케일 , width, height는 객체크기
for y, row in enumerate(pixels):           #enumerate()
    for x, v in enumerate(row):
        img.putpixel((x,y), v)
img.show()

```

binary 이미지 복사 파트 PPT 참고하기

11장 Numpy

```
a = np.array([1,2,3], dtype='int')    #array([1, 2, 3])
a = np.array([1,2,3], dtype='float')  #array([1., 2., 3.])
```

```
A = np.array([[1,2],[3,4]])
B = np.matrix([[1,2],[3,4]])
```

```
A
array([[1, 2],
       [3, 4]])
```

```
B
matrix([[1, 2],
        [3, 4]])
```

```
C = np.array([[3,4],[5,6]])
D = np.matrix([[3,4],[5,6]])
```

```
A.dot(C)                #dot()을 쓰면 array 끼리 행렬곱
array([[13, 16],
       [29, 36]])
```

```
B*D                      #matrix끼리 곱하면 행렬곱
matrix([[13, 16],
        [29, 36]])
```

```
A*C                      #그냥 array끼리 곱하면 원소끼리 곱함
array([[ 3,  8],
       [15, 24]])
```

```
matB = np.array([[3,1,9],[2,5,8],[4,7,6]])    #3x3 행렬 생성
matC = np.array(((3,1,9),(2,5,8),(4,7,6)))
```

```
np.zeros((2,3))          #2행3열의 원소가 0인 행렬 생성
array([[0., 0., 0.],
       [0., 0., 0.]])
```

```
np.ones((2,3))           #2행3열의 원소가 1인 행렬 생성
array([[1., 1., 1.],
       [1., 1., 1.]])
```

```
np.arange(10, 20, 2)      #array([10, 12, 14, 16, 18])    range와 비슷.
```

```
val = 0
l = []
for i in range(3):
    row = []
    for j in range(3):
        row.append(val)
        val += 1
    l.append(row)
ans = np.array(l)
print(ans)
```

```
[[0 1 2]
 [3 4 5]
 [6 7 8]]
```

```
l = []
for i in range(5):
    row = []
    val = 0
    for j in range(5):
        row.append(val)
        val += 1
    l.append(row)
ans = np.array(l)
print(ans)
```

```
[[0 1 2 3 4]
 [0 1 2 3 4]
 [0 1 2 3 4]
 [0 1 2 3 4]
 [0 1 2 3 4]]
```

np.arange(9).reshape((3,3)) #0부터 8까지 만들고 (3,3)으로 reshape해라

```
array([[0, 1, 2],
       [3, 4, 5],
       [6, 7, 8]])
```

```
x = np.array([1.1, 2.1, 3.1], dtype='float')    #float로 데이터타입 지정
dt = np.dtype('float')                          #데이터 타입 객체
x = np.array([1.1, 2.1, 3.1], dtype=dt)
```

```
matB = np.array([[3,1,9],[2,5,8],[4,7,6]])
matB.shape                      #3,3
matB.mean()                    #5.0    평균 출력
matB.ndim                      #2      차원
mat.shape?                    #----->
```

```
array([[3, 1, 9],
       [2, 5, 8],
       [4, 7, 6]])
```

```
Type:            tuple
String form:    (3, 3)
Length:         2
Docstring:
Built-in immutable sequence.
```

```
A = np.array([[-2,1],[1.5,-0.5]])
B = np.array([[3,4],[0.5, 1]])
```

If no argument is given, the constructor returns an empty tuple.
If iterable is specified the tuple is initialized from iterable's

Array 연산

If the argument is a tuple, the return value is the same object.

```
A        array([[ -2. ,  1. ],
               [ 1.5, -0.5]])
```

```
A%B      array([[ 1. ,  1. ],
               [ 0. ,  0.5]])
```

```
B        array([[ 3. ,  4. ],
               [ 0.5,  1. ]])
```

```
A.dot(B)   array([[ -5.5 , -7. ],
                 [ 4.25,  5.5 ]])
```

```
A+B      array([[ 1. ,  5. ],
               [ 2. ,  0.5]])
```

```
np.dot(A,B) array([[ -5.5 , -7. ],
                 [ 4.25,  5.5 ]])
```

```
A-B      array([[ -5. , -3. ],
               [ 1. , -1.5]])
```

```
np.linalg.inv(A)    array([[ 1. ,  2. ],
                 [ 3. ,  4. ]])
역행렬
```

```
A*B      array([[ -6. ,  4. ],
               [ 0.75, -0.5 ]])
```

```
np.linalg.det(A)                      0.5
determinant
```

```
A/B      array([[ -0.66666667,  0.25 ],
               [ 3. , -0.5 ]])
```

```
np.linalg.matrix_rank(A)              2
랭크
```

```
A**B     array([[ -8. ,  1. ],
               [ 1.22474487, -0.5 ]])
```

```
A//B     array([[ -1. ,  0. ],
               [ 3. , -1. ]])
```

```
a = [1,2,3,4]
a[2]          #3
x2d = np.arange(12).reshape(3,4)
```

```
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
```

```
a = x2d[1][1:]          #array([5, 6, 7])
a[0]==-1                #array([-1,  6,  7])
```

슬라이싱
원본에도 수정이 영향을 준다!

```
x2d
array([[ 0,  1,  2,  3],
       [ 4, -1,  6,  7],
       [ 8,  9, 10, 11]])
```

```
b = x2d[1:,2:]
b
array([[ 6,  7],
       [10, 11]])
```

슬라이싱
원본에도 수정이 영향을 준다!

```
b[0][0]==-10
x2d
array([[ 0,  1,  2,  3],
       [ 4, -1, -10,  7],
       [ 8,  9, 10, 11]])
```

```
b[:,:] = 20
b
array([[20, 20],
       [20, 20]])
```

```
x2d
array([[ 0,  1,  2,  3],
       [ 4, -1, 20, 20],
       [ 8,  9, 20, 20]])
```

```
dat = np.random.rand(2,3)
dat
array([[0.66380302, 0.69339641, 0.10517152],
       [0.35933505, 0.88009337, 0.64704791]])
```

```
bmask = dat>0.5          #부등호로 하면 TF로 반환!
bmask
array([[ True,  True, False],
       [False,  True, False]])
```

```
highd = dat[bmask]        #bmask에 해당되는것만 남겨라
highd
array([0.55681399, 0.87065304, 0.9845811 ])
```

```
highd[0]=1                복사한 것이라 원본에 영향을 미치지 않음
dat
array([[0.69898683, 0.42702343, 0.34048698],
       [0.27794034, 0.92174483, 0.80708481]])
```

케이스	코드 예시	동작 방식	원본 <code>x2d/dat</code> 영향	
☒ 뷰(View)	<pre>b = x2d[1:, 2:] b[0][0] = -10</pre>	참조(같은 메모리 공유)	○ (원본 <code>x2d</code> 변경됨)	슬라이싱 팬시 인덱싱
☒ 복사(Copy)	<pre>highd = dat[dat > 0.5] highd[0] = 1</pre>	복사본(새 메모리)	✗ (원본 <code>dat</code> 그대로 유지됨)	콜론 : 리스트나 배열

```
nda = np.arange(10)
nda
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
ndb = nda[1:4]          #array([1, 2, 3])
ndc = nda[[1,2,3]]      #array([1, 2, 3])
```

슬라이싱
원본에도 수정이 영향을 준다!
팬시 인덱싱
원본에 영향을 미치지 않음

```
ndc[0]==-10
nda
#array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
ndb[0]==-10
nda
#array([ 0, -10,  2,  3,  4,  5,  6,  7,  8,  9])
```

```
nda = np.arange(12).reshape(3,4)
```

```
nda
```

```
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
```

```
ndb = nda[[0,2,1],[2,0,3]]
```

```
ndb
```

```
array([2, 8, 7])
```

```
ndb[0] = -10
```

```
nda
```

```
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
```

```
nda[np.ix([0,2],[1,3])] 행 0번 2번, 열 1번 3번만 팬시 인덱싱해라
```

```
array([[ 1,  3],
       [ 9, 11]])
```

```
np.square(nda)
```

모든 요소를 제곱해라

```
array([[ 0,  1,  4,  9],
       [16, 25, 36, 49],
       [64, 81, 100, 121]])
```

```
for i in range(3):
```

```
    for j in range(4):
```

```
        nda[i][j] = nda[i][j]**2
```

```
array([[ 0,  1,  4,  9],
       [16, 25, 36, 49],
       [64, 81, 100, 121]])
```

```
nda = np.array([1,2,3])
```

```
nda+1
```

```
#array([2, 3, 4])
```

스칼라합

```
nda+np.array([1,1,1])
```

```
#array([2, 3, 4])
```

배열합

```
nda = np.arange(24).reshape(4,3,2)
```

```
ndb = np.arange(6).reshape(3,2)
```

```
ndc = np.arange(3).reshape(3,1)
```

```
res = nda+ndb-ndc
```

```
array([[[ 0,  2],
        [ 3,  5],
        [ 6,  8]],
```

```
res.shape # (4, 3, 2)
```

```
ndb.shape # (3, 2) -> (4,3,2)
```

```
ndc.shape # (3, 1) -> (4,3,2)
```

```
nda+ndb-ndc #----->
```

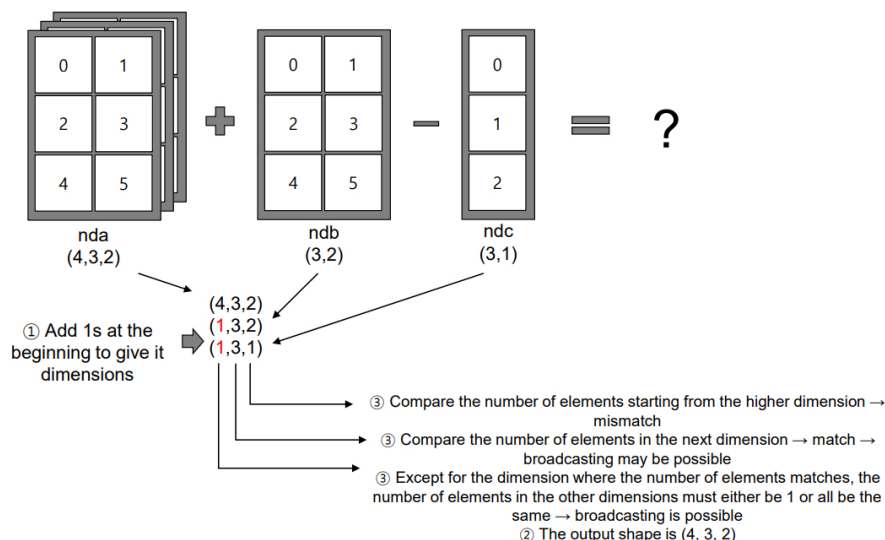
```
[[ 6,  8],
 [ 9, 11],
 [12, 14]],
```

```
[[12, 14],
 [15, 17],
 [18, 20]],
```

차원이 확장된다(broadcasting)

> broadcasting을 하려면 차원수와 크기가 둘 다 맞아야 한다. 불가능하면 차원을 바꿔야 함.

```
[[18, 20],
 [21, 23],
 [24, 26]]])
```



```
nda = np.arange(12).reshape(3,4)
ndb = np.arange(4)

ndb          #array([0, 1, 2, 3])
ndb.shape    #[4,]          1차원

np.array([[0, 1, 2, 3]]).shape  #(1, 4)          2차원

nda+ndb
      array([[ 0,  2,  4,  6],
             [ 4,  6,  8, 10],
             [ 8, 10, 12, 14]])

ndb = np.arange(4).reshape(4,1)
ndb.shape    #(4, 1)
ndb          -----> array([[0],
                             [1],
                             [2],
                             [3]])

nda+ndb.reshape(1,4)  array([[ 0,  2,  4,  6],
                             [ 4,  6,  8, 10],
                             [ 8, 10, 12, 14]])

>> 이런식으로 연산을 하려면 모양을 바꿔줘야 함.
```

```
A = np.array([[1],[2],[3]])
B = np.array([[1,2,3,4]])
A.shape      #(3,1)
B.shape      #(1,4)

A+B
      array([[2, 3, 4, 5],
             [3, 4, 5, 6],
             [4, 5, 6, 7]])

C = A.reshape((1,3,1))+B.reshape((1,1,4))
C.shape      (1,3,4)
C.reshape((3,4))
      array([[2, 3, 4, 5],
             [3, 4, 5, 6],
             [4, 5, 6, 7]])
```

12장 Pandas

```
import numpy as np
import pandas as pd

data_frame = pd.read_csv("example_csv.csv")    #example_csv.csv 파일 불러오기
data_frame

data_frame = pd.read_excel("example_excel.xlsx", header=None, sheet_name=1)

raw_data = {'col0':[1,2,3,4], 'col1':[5,6,7,8], 'col2':[9,10,11,12]}
pd.DataFrame(raw_data)
```

	AA	BB	CC
0	1	4	10
1	2	5	11
2	3	6	12

	col0	col1	col2
0	1	5	9
1	2	6	10
2	3	7	11
3	4	8	12

```
data_frame = pd.DataFrame([[1,2,3],[4,5,6],[7,8,9],[10,11,12]])
```

```
data_frame
```

	0	1	2
0	1	2	3
1	4	5	6
2	7	8	9
3	10	11	12

```
raw_data = np.array([[1,2,3],[4,5,6],[7,8,9],[10,11,12]])
```

```
data_frame = pd.DataFrame(raw_data)
```

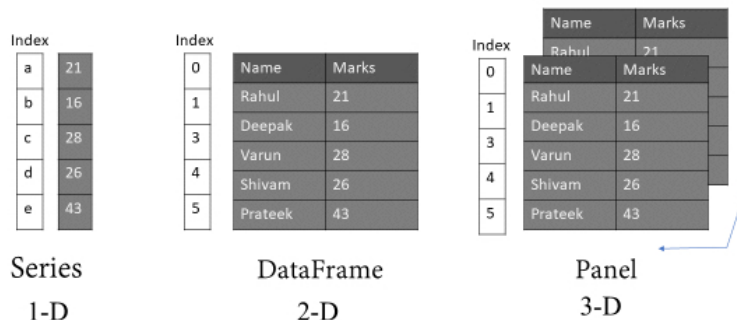
```
raw_data
```

```
array([[ 1,  2,  3],
       [ 4,  5,  6],
       [ 7,  8,  9],
       [10, 11, 12]])
```

	0	1	2
0	1	2	3
1	4	5	6
2	7	8	9
3	10	11	12

- ① Series (suitable for 1-dimensional data)
- ② DataFrame (suitable for 2-dimensional data)
- ③ Panel (suitable for 3-dimensional data)

판다스에서 데이터 타입



1차원

```
data_frame.to_csv("example_output.csv", sep=',', index=False) #콤마로 구분. 인덱스는 빼고
```

```
dat = pd.Series([1,2,3,4])
```

```
dat
```

```
0    1
1    2
2    3
3    4
```

```
dat = pd.Series(np.array([1,2,3,4]))
```

```
dat.values #array([1, 2, 3, 4])
```

```
dtype: int32
```

```
dat = pd.Series([1,2,3,4])
```

```
dat.values #array([1, 2, 3, 4], dtype=int64)
```

```
pd.Series(['aa', 'bb', 'c', 'd'])
```

```
0    aa
1    bb
2     c
3     d
dtype: object
```

```
pd.Series([1, 'bb', 3.4, 'd'])
```

```
0    1
1    bb
2    3.4
3     d
dtype: object
```

```
dat5 = pd.Series([1,3,6,12], index=[1,10,20,33])
```

#인덱스 직접 지정!

```
dat5
```

```
dat5[33] #12
```

```
1    1
10   3
20   6
33  12
dtype: int64
```

```
dat7 = pd.Series({'a':1, 'b':3, 'c':6, 'd':12})
```

```
dat7
```

```
a    1
b    3
c    6
d   12
dtype: int64
```



```
dat7.index = ['abc', 'bcd', 'cde', 'def']    #인덱스 업데이트
dat7
```

```
abc      1
bcd      3
cde      6
def     12
dtype: int64
```

2차원

```
data = [[1,2],[4,5]]
data_frame = pd.DataFrame(data, index=['row1', 'row2'], columns=['col1', 'col2'])
```

	col1	col2
row1	1	2
row2	4	5

```
data_frame.index      #Index(['row1', 'row2'], dtype='object')
data_frame.columns    #Index(['col1', 'col2'], dtype='object')
```

```
data_frame.index = ['row3', 'row4']    #인덱스 업데이트
data_frame
```

	col1	col2
row3	1	2
row4	4	5

```
data_frame.columns = ['new_col1', 'new_col2']    #column 업데이트
data_frame
```

	new_col1	new_col2
row3	1	2
row4	4	5

```
data_frame = pd.DataFrame({              #DataFrame 통째로 업데이트
    "Age" : [20, 30, 45, 25, 31],
    "Height" : [180, 170, 160, 150, 140],
    "Sex" : ["m", "m", "f", "f", "f"]
})
data_frame
```

	Age	Height	Sex
0	20	180	m
1	30	170	m
2	45	160	f
3	25	150	f
4	31	140	f

```
data_frame["Age"]
```

```
0    20
1    30
2    45
3    25
4    31
Name: Age, dtype: int64
```

```
data_frame[["Age", "Height"]]
```

	Age	Height
0	20	180
1	30	170
2	45	160
3	25	150
4	31	140

```
data_frame.iloc[1]    #int location
                      #n번째 행을 출력
```

```
Age      30
Height   170
Sex       m
Name: 1, dtype: object
```

```
data_frame.iloc[2:4]
```

	Age	Height	Sex
2	45	160	f
3	25	150	f

	Age	Height	Sex
1	30	170	m
2	45	160	f
3	25	150	f
4	31	140	f

```
data_frame.iloc[1:] ----->
```

data_frame.sort_values(by='Age', ascending=False) #내림차순 정렬

	Age	Height	Sex
2	45	160	f
4	31	140	f
1	30	170	m
3	25	150	f
0	20	180	m

data_frame.loc[2, "Height"] #160 정확한 값 출력 [행,열]

data_frame.loc[1:3, "Height"]

1	170
2	160
3	150

Name: Height, dtype: int64

data_frame.loc[[0, 2, 4], "Height"]

0	180
2	160
4	140

Name: Height, dtype: int64

data_frame[data_frame["Age"] <= 30]

	Age	Height	Sex
0	20	180	m
1	30	170	m
3	25	150	f

data_frame[(data_frame["Age"]>=25) & (data_frame["Age"]<=30)]

	Age	Height	Sex
1	30	170	m
3	25	150	f

data_frame

	Age	Height	Sex
0	20	180	m
1	30	170	m
2	45	160	f
3	25	150	f
4	31	140	f

data_frame["Weight"] = [50,60,70,80,90]

	Age	Height	Sex	Weight
0	20	180	m	50
1	30	170	m	60
2	45	160	f	70
3	25	150	f	80
4	31	140	f	90

data_frame = pd.DataFrame({ #재정의
 "Age" : [20, 30, 45, 25, 31],
 "Height" : [180, 170, 160, 150, 140],
 "Sex" : ["m", "m", "f", "f", "f"]
})

data_frame

	Age	Height	Sex
0	20	180	m
1	30	170	m
2	45	160	f
3	25	150	f
4	31	140	f

data_frame.loc[5] = [15, 150, "m"]

data_frame ----->

len(data_frame) #6

	Age	Height	Sex
0	20	180	m
1	30	170	m
2	45	160	f
3	25	150	f
4	31	140	f
5	15	150	m

```
data_frame = pd.DataFrame({
    "Age" : [20, 30, 45, 25, 31],
    "Height" : [180, 170, 160, 150, 140],
    "Sex" : ["m", "m", "f", "f", "f"]
})
data_frame
```

	Age	Height	Sex
0	20	180	m
1	30	170	m
2	45	160	f
3	25	150	f
4	31	140	f

```
data_frame.drop(["Height"], axis=1)
★axis=1은 열을 삭제하겠다는 의미
```

	Age	Sex
0	20	m
1	30	m
2	45	f
3	25	f
4	31	f

```
data_frame.drop(3, axis=0)
```

	Age	Height	Sex
0	20	180	m
1	30	170	m
2	45	160	f
4	31	140	f

```
data_frame = pd.DataFrame({
    "Age" : [20, 30, 45, 25, 31],
    "Height" : [180, 170, 160, 150, 140],
    "Sex" : ["m", "m", "f", "f", "f"]
})
```

```
pd.concat([data_frame, data_frame], axis=0)
```

행 아래로 이어붙인다

	Age	Height	Sex
0	20	180	m
1	30	170	m
2	45	160	f
3	25	150	f
4	31	140	f
0	20	180	m
1	30	170	m
2	45	160	f
3	25	150	f
4	31	140	f

```
pd.concat([data_frame, data_frame], axis=1)
```

열 옆으로 이어붙인다

	Age	Height	Sex	Age	Height	Sex
0	20	180	m	20	180	m
1	30	170	m	30	170	m
2	45	160	f	45	160	f
3	25	150	f	25	150	f
4	31	140	f	31	140	f

```
data_frame["Age"].std()
```

표준편차를 구해라

```
data_frame = pd.DataFrame({
    "Age" : [20, 30, 45, 25, 31],
    "Height" : [180, 170, 160, 150, 140],
    "Sex" : ["m", "m", "f", "f", "f"]
})
```

```
avg_m = data_frame[(data_frame["Age"]>=30) & (data_frame["Sex"]=="m")]["Height"].mean() #175.0
avg_f = data_frame[(data_frame["Age"]>=30) & (data_frame["Sex"]=="f")]["Height"].mean() #150.0
print(avg_m, avg_f)
```

```
df = pd.DataFrame({
    'Name': ['Tom', 'Jerry', 'Mickey', 'Donald', 'Goofy'],
    'Score': [92, 85, 97, 88, 75]
})
```

Score를 기준으로 내림차순해서 3명의 "Name"만 top_names에 저장

```
top_names = df.sort_values(by='Score', ascending=False).head(3)["Name"]
top_names.tolist()          #['Mickey', 'Tom', 'Donald']
```

```
df = pd.DataFrame({
    'Name': ['Anna', 'Ben', 'Cara', 'Dan', 'Ella'],
    'Salary': [4800, 5100, 5300, 4700, 6000]
})
```

```
SalaryLevel = []
for i in range(len(df)):
    if df.loc[i, "Salary"] >= 500:
        SalaryLevel.append("High")
    else:
        SalaryLevel.append("Low")
df["SalaryLevel"] = SalaryLevel
print(df)
```

	Name	Salary	SalaryLevel
0	Anna	4800	High
1	Ben	5100	High
2	Cara	5300	High
3	Dan	4700	High
4	Ella	6000	High

```
data_frame = pd.DataFrame(np.random.randn(3,4))
np.log(data_frame)          #로그계산
```

	0	1	2	3
0	NaN	-0.450654	NaN	-1.704497
1	NaN	NaN	0.309452	0.114401
2	0.068127	NaN	NaN	-0.361798

```
data_frame
```

	0	1	2	3
0	-2.926207	0.637211	-0.692911	0.181864
1	-0.496242	-0.138268	1.362678	1.121202
2	1.070501	-0.455658	-1.096164	0.696423

```
np.array(data_frame)
array([[ -2.92620711,  0.63721099, -0.69291079,  0.18186386],
       [-0.49624223, -0.13826849,  1.36267779,  1.12120215],
       [ 1.07050099, -0.4556585 , -1.09616441,  0.69642328]])
```

```
df = pd.read_csv("example_csv.csv")
```

	AA	BB	CC
0	1	4.0	10.0
1	2	NaN	11.0
2	3	6.0	NaN

```
df.dropna()          #NaN가 있는 행을 삭제해버림.
```

```
df.doropna(axis=1)   #NaN이 있는 열을 삭제해버림
```

	AA	BB	CC
0	1	4.0	10.0

```
df.dropna(subset=['BB'])      #BB열을 기준으로 그 열에 NaN이 있는 것만 삭제
```

	AA	BB	CC
0	1	4.0	10.0
2	3	6.0	NaN

```
df.dropna(axis=1, subset=[2])  #??
```

df.fillna(0)

#NaN에 0을 채운다

df.isnull().sum()

#null인 것의 개수를 셈.

```
AA    0
BB    1
CC    1
dtype: int64
```

	AA	BB	CC
0	1	4.0	10.0
1	2	0.0	11.0
2	3	6.0	0.0

df.fillna(method='ffill')

#front fill 윗 값으로 채움

df.fillna(method='bfill')

#backward fill 아랫 값으로 채움

	AA	BB	CC
0	1	4.0	10.0
1	2	4.0	11.0
2	3	6.0	11.0

Replace NaN with the value from the previous row in the same column
data_frame.fillna(method='ffill')

Replace NaN with the value from the next row in the same column
Data_frame.fillna(method='bfill')

	AA	BB	CC
0	1	4.0	10.0
1	2	6.0	11.0
2	3	6.0	NaN

13장 Matplotlib

```
import numpy as np
import matplotlib as mpl
import matplotlib.pyplot as plt
```

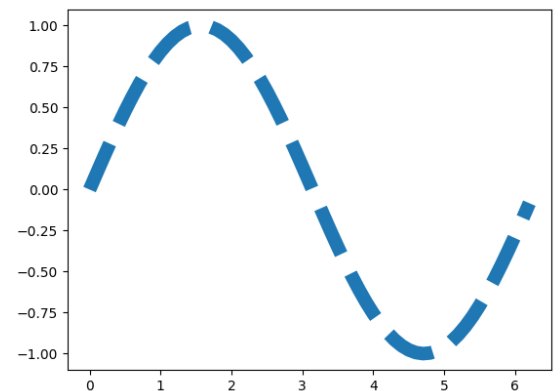
```
mpl.rcParams['lines.linewidth'] = 10      기본 선을 10으로
mpl.rcParams['lines.linestyle'] = '--'    선 스타일을 --로
```

#위 두줄 하나로!

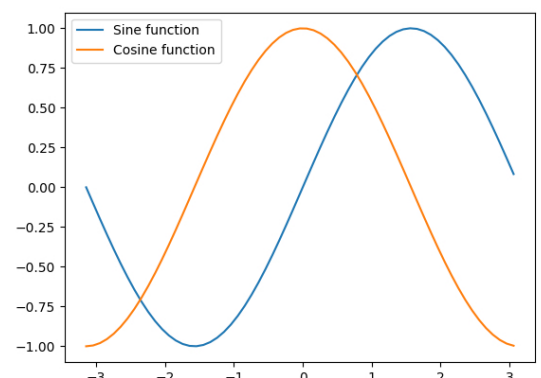
```
mpl.rc('lines', linewidth=10, linestyle='--')    선두께 10, 선스타일 --
```

```
t = np.arange(0, 2 * np.pi, 0.1)    #0부터 2π까지 0.1간격으로 만든 리스트
plt.figure(1)                        #1번 그림 생성
plt.plot(t, np.sin(t))                #x축 : t, y축 : sint
plt.show()
```

```
mpl.rcParamsdefaults()                #이걸 하면 설정을 다 기본으로 돌려버린다.
```



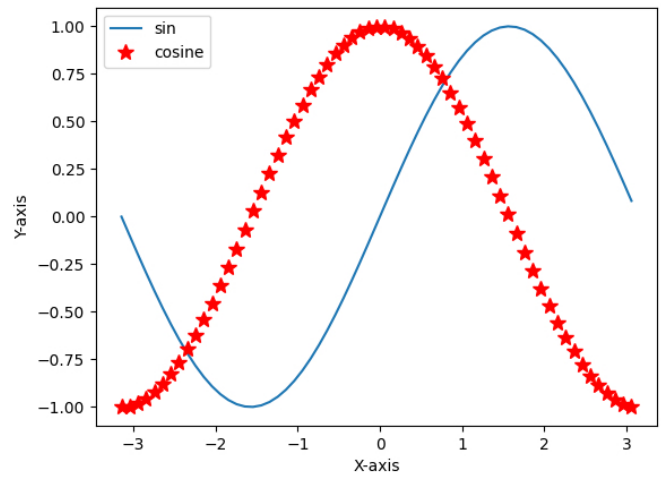
```
x = np.arange(-np.pi, np.pi, 0.1)
y1 = np.sin(x)
y2 = np.cos(x)
plt.plot(x, y1, label = 'Sine function')
plt.plot(x, y2, label = 'Cosine function')
plt.legend(('Sine function', 'Cosine function'), loc = 'best')
#범례. 순서대로 이름 표시. loc='best' 알아서
plt.show()
```



```

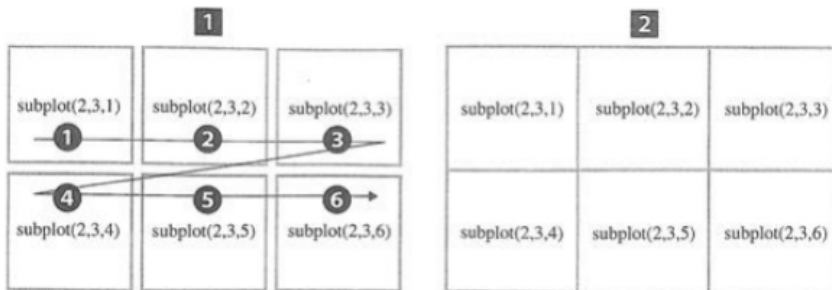
x = np.arange(-np.pi, np.pi, 0.1)
y1 = np.sin(x)
y2 = np.cos(x)
plt.figure(1)
plt.clf()          #현재 있는 그림을 모두 지워라
plt.plot(x, y1, label='sin')
plt.plot(x, y2, 'r*', markersize=10, label='cosine')
                        #마커사이즈 10번은 별표
plt.xlabel('X-axis'); plt.ylabel('Y-axis')
plt.legend(loc='best')
plt.show()

```



서브플롯

subplot(L, M, N)은 L행 M열 격자 배치 중에서 N번째 위치의 그래프를 그립니다.
subplot(2, 3, 4)는 2행 3열 중 4번째 위치 (2행 1열) 에 그린다는 뜻입니다.



```

plt.clf()
# subplot(2,3,1)
plt.subplot(2, 3, 1)
plt.xticks([]) # x축 눈금 제거
plt.yticks([]) # y축 눈금 제거
plt.text(0.5, 0.5, 'subplot(2,3,1)', ha='center', va='center', size=25)

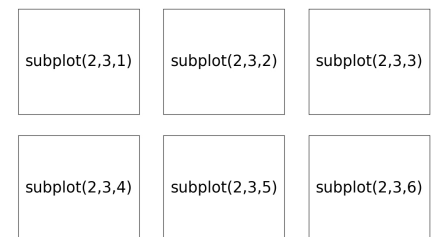
```

```

..... 중략
plt.text(0.5, 0.5, 'subplot(2,3,5)', ha='center', va='center', size=25)
# subplot(2,3,6)
plt.subplot(2, 3, 6)
plt.xticks([])
plt.yticks([])
plt.text(0.5, 0.5, 'subplot(2,3,6)', ha='center', va='center', size=25)

plt.show()

```



더 복잡한 것이 필요하면 GridSpec()

```

import matplotlib.gridspec as gs
G = gs.GridSpec(3,3)
axes_1 = plt.subplot(G[0, :])

```

```
import matplotlib.gridspec as gs
```

```
plt.figure()
```

```
G = gs.GridSpec(3, 3)
```

```
axes_1 = plt.subplot(G[0, :])    #첫번째행 전부 사용
plt.xticks([]); plt.yticks([])
plt.text(0.5, 0.5, 'subplot 1', ha='center', va='center', size=22)
```

```
axes_2 = plt.subplot(G[1, :-1])  #두번째행 마지막제외 사용
x = np.arange(-np.pi, np.pi, 0.1)
y = np.sin(x)
plt.plot(x, y)
plt.text(-0.5, 0, 'subplot 2', ha='center', va='center', size=22)
```

```
axes_3 = plt.subplot(G[1:, -1])  #2행부터 끝까지, 마지막 열
plt.xticks([]); plt.yticks([])
plt.text(0.5, 0.5, 'subplot 3', ha='center', va='center', size=22)
```

```
axes_4 = plt.subplot(G[-1, 0])   # 마지막 행, 첫번째 열
plt.xticks([]); plt.yticks([])
plt.text(0.5, 0.5, 'subplot 4', ha='center', va='center', size=22)
```

```
axes_5 = plt.subplot(G[-1, -2])  #마지막 행, 뒤에서 두번째 열
plt.xticks([]); plt.yticks([])
plt.text(0.5, 0.5, 'subplot 5', ha='center', va='center', size=22)
plt.show()
```

```
plt.subplots_adjust(wspace = 0, hspace = 0)
```

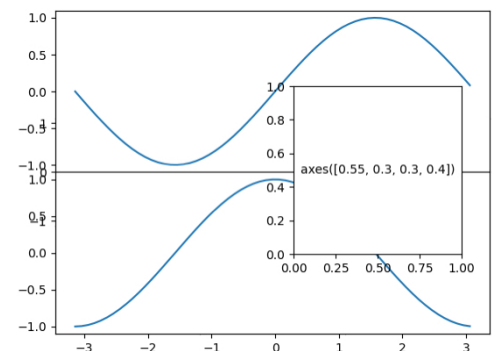
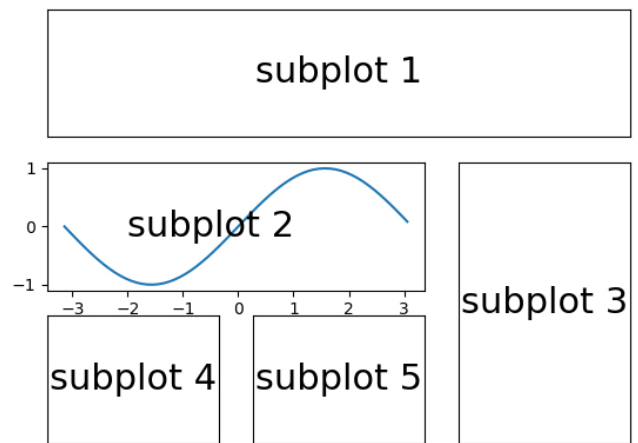
#그래프 간 간격 조절

그리드 상관없이 아무데나 그래프 띄워버리기

```
plt.figure(1)
plt.subplot(2,1,1)
x = np.arange(-np.pi, np.pi, 0.1)
plt.plot(x, np.sin(x))
plt.subplot(2,1,2)
plt.plot(x, np.cos(x))
```

```
plt.axes([0.55, 0.3, 0.3, 0.4])    # (left, bottom, width, height)
plt.text(0.5,0.5, 'axes([0.55, 0.3, 0.3, 0.4])', ha='center', va='center')
```

```
plt.show()
```

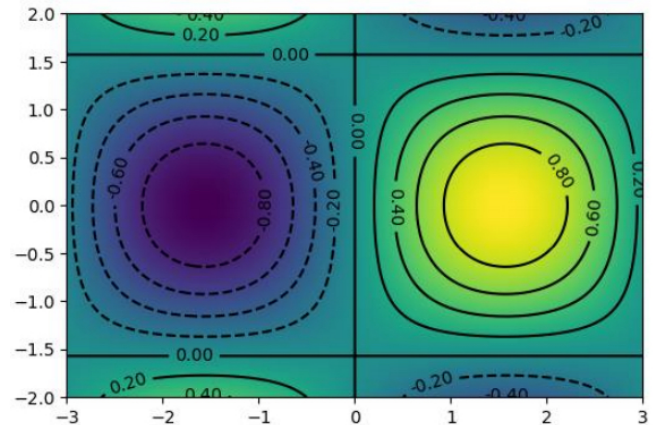


Contour Chart 등고선 차트

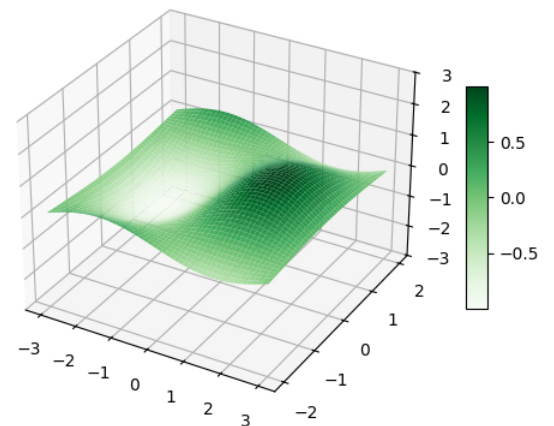
```
N = 200
x = np.linspace(-3.0, 3.0, N)
y = np.linspace(-2.0, 2.0, N)
X, Y = np.meshgrid(x, y)
Z = np.sin(X) * np.cos(Y)
```

-3부터 3까지 균등하게 200개의 점 생성
-2부터 2까지 균등하게 200개의 점 생성
2차원 격자 좌표 생성

```
plt.figure(figsize=(6, 4))
im = plt.imshow(
    Z,
    interpolation='bilinear',
    origin='lower',
    extent=[-3, 3, -2, 2],
    cmap='viridis'
)
levels = np.linspace(-1, 1, 11)
ctr = plt.contour(
    X, Y, Z,
    levels=levels,
    colors='k',
    linewidths=1.5
)
plt.clabel(ctr, inline=True, fmt='%1.2f', fontsize=10)
plt.show()
```



```
fig = plt.figure(1)
ax = fig.add_subplot(111, projection='3d')
surf = ax.plot_surface(X, Y, Z, cmap=cm.Greens)
ax.set_zlim3d(-3.01, 3.01)
plt.colorbar(surf, shrink=0.5, aspect=10)
plt.show()
```



14장 Object Oriented Programming

Procedural :

Object-Oriented :

OOP 5원칙

SRP Single Responsibility Principle (단일 책임 원칙): 클래스는 하나의 역할만 책임져야 한다.

OCP Open-Closed Principle (개방-폐쇄 원칙): 코드는 수정 없이 확장 가능해야 한다.

LSP Liskov Substitution Principle (리스코프 치환 원칙): 자식 클래스는 부모 클래스처럼 사용될 수 있어야 한다.

ISP Interface Segregation Principle (인터페이스 분리 원칙): 인터페이스는 목적에 따라 작게 나누어야 한다.

DIP Dependency Inversion Principle (의존 역전 원칙): 구체 클래스보다 추상(인터페이스)에 의존해야 한다.


```
class Car:                                #클래스 선언
    # attributes
    color = ""
    speed = 0

    # methods
    def upSpeed(self, amt):                #메서드에는 꼭 self 가 있어야 함.
        self.speed += amt
    def downSpeed(self, amt):
        self.speed -= amt
```

```
car1 = Car()          #인스턴스 생성
car2 = Car()
car3 = Car()
```

```
car1.color = "red"
car1.speed = 0
car2.color = "blue"
car2.speed = 0
car3.color = "yellow"
car3.speed = 0
car1.upSpeed(30)
car2.downSpeed(60)
```

```
car1.speed    #30
car2.speed    #-60
```

```
class Rectangle:
    width = 0
    height = 0
    def area(self):
        return self.width * self.height
```

```
Rectangle1 = Rectangle()          #인스턴스 생성
```

```
Rectangle1.width = 10
Rectangle1.height = 5
```

```
print(Rectangle1.area())          #50
```

상속

```
class Car:
    # attributes
    color = ""
    speed = 0
    # methods
    def upSpeed(self, amt):
        self.speed += amt
    def downSpeed(self, amt):
        self.speed -= amt
```

```
class PassengerCar(Car):                                #상속! Inheretance
    # attributes
    num_of_seats = 0
    # methods
    def check_number_of_seats(self):
        return self.num_of_seats
```

```
class Truck(Car):
    # attributes
    cargo_capacity = 0
    # methods
    def check_cargo_capacity(self):
        return self.cargo_capacity
```

```
sedan1 = PassengerCar()
sedan1.color
```

```
class Car:
    speed = 0
    def upSpeed(self, value):
        self.speed += value
```

```
class PassengerCar(Car):
    def upSpeed(self, value):                #메서드 오버라이딩
        self.speed += value
        if self.speed > 150:
            self.speed = 150
```

```
sedan2 = PassengerCar()
sedan2.speed = 200
sedan2.upSpeed(50)
sedan2.speed          #150
```

```
class Person:
    name = ""
    def introduce(self):
        print(self.name)
class Student(Person):
    student_id = 0
    def introduce(self):
        print(self.name, self.student_id)
```

```
std1 = Student()
std1.name = "Son"
std1.student_id = 1001
std1.introduce()
```

```

class Car:
    def __init__(self, color, speed):          # init 객체를 만들 때 자동으로 호출됨
        self.color = color
        self.speed = speed
    # methods
    def upSpeed(self, amt):
        self.speed += amt
    def downSpeed(self, amt):
        self.speed -= amt
    def __repr__(self):                        # string을 리턴함.
        return self.color
    def __add__(self, other):                  # 연산자 오버로딩      + 연산자를 사용할 때 동작
                                                # 더하기를 하면 실제로는 이게 호출됨
        return self.speed + other.speed
    def __sub__(self, other):                  # 연산자 오버로딩      - 연산자를 사용할 때 동작
        return self.speed - other.speed
    def __lt__(self, other):                  # 연산자 오버로딩      < 연산자를 사용할 때 동작
        return self.speed < other.speed

```

```

car1 = Car("red", 10)
car2 = Car("blue", 20)

```

```

car1 < car2      #True
car1 == car2     #False
car1 - car2      #-10

```

```

car1.__add__(car2)    #30           이렇게 직접 사용하는것도 가능.
print(car1)           #red         __repr__이 호출됨

```

```

class Score:
    def __init__(self, name, kor, eng, math):
        self.name = name
        self.kor = kor
        self.eng = eng
        self.math = math

    def get_avg(self):
        return (self.kor + self.eng + self.math) / 3

    def __repr__(self):
        return str(self.get_avg())

    def __add__(self, other):
        return self.get_avg() + other.get_avg()

    def __lt__(self, other):
        return self.get_avg() < other.get_avg()

```

```
s1 = Score("Kim", 80, 90, 85)
s2 = Score("Lee", 10, 20, 30)
```

```
print(s1)          #85.0
s1+s2              #105.0
```

```
l = [s1, s2]
print(sorted(l))    #[20.0, 85.0]
sorted(l)           #[20.0, 85.0]
l                   #[85.0, 20.0]
```